Lab 2 – Språkteknologi (5DV218)

Text Classification of the 20 Newsgroups Dataset

Name: Michail Pettas | **Course:** 5DV218 Språkteknologi (Umeå University) | **Code file:** lab2.py

1. Dataset Preparation (Questions 1 & 2)

In this lab, I worked with the 20 Newsgroups dataset, a benchmark text collection consisting of Usenet postings distributed across 20 distinct discussion categories. I loaded the dataset using scikit-learn's fetch_20newsgroups function, separating it into the standard training (subset='train') and test (subset='test') subsets.

Answers to Questions 1 and 2:

• **Question 1 (Instance count):** The dataset contains a total of 18,846 instances, with 11,314 documents in the training subset and 7,532 documents in the test subset. This corresponds to an approximate 60% training / 40% testing chronological split.

• **Question 2 (Classes):** There are 20 classes in the dataset. Table 1 lists all category names and their document counts in both subsets.

ID	Class Name (Newsgroup)	Train Instances	Test Instances
0	alt.atheism	480	319
1	comp.graphics	584	389
2	comp.os.ms-windows.misc	591	394
3	comp.sys.ibm.pc.hardware	590	392
4	comp.sys.mac.hardware	578	385
5	comp.windows.x	593	395
6	misc.forsale	585	390
7	rec.autos	594	396
8	rec.motorcycles	598	398
9	rec.sport.baseball	597	397
10	rec.sport.hockey	600	399
11	sci.crypt	595	396
12	sci.electronics	591	393
13	sci.med	594	396
14	sci.space	593	394
15	soc.religion.christian	599	398
16	talk.politics.guns	546	364
17	talk.politics.mideast	564	376
18	talk.politics.misc	465	310
19	talk.religion.misc	377	251
	Total	**11,314**	**7,532**

Table 1. Distribution of instances across all 20 newsgroups in training and test sets.

As shown in Table 1, the classes are fairly balanced, with most groups containing around 550–600 training instances and 380–400 test instances. The only notable exceptions are the general discussion groups, specifically talk.religion.misc (377 train / 251 test) and talk.politics.misc (465 train / 310 test), which are somewhat smaller.

2. Text Preprocessing & Vocabulary Representation (Question 3)

To prepare the raw texts for vectorization, I implemented the function preprocess_text() using NLTK. The preprocessing pipeline executes the following steps in sequence:
1. Case Normalization: Lowercasing all characters with text.lower(), ensuring capitalized words (e.g. 'The' at sentence starts) are matched during stop-word removal.
2. Tokenization: Splitting text into tokens using NLTK's word_tokenize().
3. Non-letter Filtering: Retaining only tokens where token.isalpha() is True. This eliminates punctuation, standalone digits, mixed alphanumeric tokens, email header artifacts, IP addresses, and hostnames (e.g., 'wam.umd.edu' or '2-door').
4. Stop-word Removal: Filtering out common English function words using NLTK's stopwords corpus (179 words).
5. Stemming (optional): Applying NLTK's SnowballStemmer('english') to conflate morphological variants into their root stems.

As recommended in the lab instructions, the preprocessed texts were dynamically stored as new attributes on the dataset objects: preprocess_data (without stemming) and preprocess_stem_data (with stemming). I evaluated four distinct preprocessing approaches:
• Raw texts: The uncleaned posts, tokenized using CountVectorizer's default alphanumeric regex pattern (\b\w\w+\b).
• Preprocessed (no stemming): Lowercased, letters-only tokens without stop words, split on whitespace.
• Preprocessed (with stemming): The above representation with Snowball stemming applied.
• Preprocessed + bigrams: The stemmed representation including both unigrams and bigrams (ngram_range=(1, 2)).

Preprocessing Approach	Vocabulary Size (Features)	Relative Change vs. Raw
Raw texts	130,107	Baseline (100.0%)
Preprocessed (no stemming)	73,266	-43.7%
Preprocessed (with stemming)	55,108	-57.6%
Preprocessed + bigrams	877,324	+574.3%

Table 2. Vocabulary dimensions extracted from the training set across the four preprocessing approaches.

Explanation of vocabulary variation (Question 3):
• Raw to Preprocessed (-43.7%): Moving from raw text (130,107) to preprocessed text without stemming (73,266) cuts the vocabulary nearly in half. Raw Usenet posts contain massive numbers of non-letter tokens: email headers (From, Lines, NNTP-Posting-Host), version strings, numbers, phone numbers, and web hostnames. Removing these via isalpha() and dropping English stop words strips out over 56,000 noisy, non-generalizable terms.
• Stemming (-57.6% vs. raw, -24.8% vs. unstemmed): Applying the Snowball stemmer collapses inflected forms (plurals, verb tenses, participles) into shared root stems (e.g., 'wondering', 'wondered', 'wonders' -> 'wonder'), reducing the feature count from 73,266 to 55,108.
• Bigrams (+574.3%): Adding bigrams to the stemmed text causes the vocabulary to explode to 877,324 features. Because any consecutive pair of words creates a distinct feature, the combinatorial space of pairs is vastly larger than the set of unigrams alone. Note that the TF-IDF representations use the exact same feature vocabularies and dimensions as their Count counterparts.

3. Supervised Classification & Model Evaluation (Question 4)

I evaluated three supervised machine learning algorithms across all eight feature representations (4 preprocessing versions × 2 vectorizers: CountVectorizer and TfidfVectorizer), yielding 24 experimental configurations in total:

• Multinomial Naive Bayes (MultinomialNB): Probabilistic classifier based on Bayes' theorem with default Laplace smoothing (α = 1.0).

• Linear Support Vector Machine (LinearSVC): Margin-based linear classifier using squared hinge loss and C = 1.0. The maximum iterations were raised to 20,000 with a fixed random seed (random_state=0) to guarantee clean convergence across all feature sets.

• Logistic Regression (LogisticRegression): Multiclass logistic regression with L2 penalty, C = 1.0, and the lbfgs solver (max_iter=3000).

Approach	Vec	Classifier	Accuracy	Precision	Recall	F1 (macro)	Time (s)
Raw texts	Count	Multinomial NB	0.7728	0.7622	0.7636	0.7451	0.1
Raw texts	Count	Linear SVM	0.7864	0.7823	0.7797	0.7792	14.5
Raw texts	Count	Logistic Regression	0.7898	0.7863	0.7827	0.7825	111.6
Raw texts	TF-IDF	Multinomial NB	0.7739	0.8255	0.7565	0.7558	0.1
Raw texts	TF-IDF	Linear SVM	0.8532	0.8514	0.8458	0.8465	1.3
Raw texts	TF-IDF	Logistic Regression	0.8278	0.8293	0.8184	0.8197	16.3
Preprocessed (no stemming)	Count	Multinomial NB	0.7970	0.7964	0.7877	0.7716	0.0
Preprocessed (no stemming)	Count	Linear SVM	0.7829	0.7812	0.7756	0.7762	7.5
Preprocessed (no stemming)	Count	Logistic Regression	0.7916	0.7914	0.7842	0.7854	25.0
Preprocessed (no stemming)	TF-IDF	Multinomial NB	0.8105	0.8414	0.7935	0.7908	0.1
Preprocessed (no stemming)	TF-IDF	Linear SVM	0.8443	0.8422	0.8363	0.8370	0.9
Preprocessed (no stemming)	TF-IDF	Logistic Regression	0.8302	0.8312	0.8197	0.8214	13.1
Preprocessed (with stemming)	Count	Multinomial NB	0.7912	0.7799	0.7821	0.7653	0.1
Preprocessed (with stemming)	Count	Linear SVM	0.7771	0.7731	0.7690	0.7691	8.5
Preprocessed (with stemming)	Count	Logistic Regression	0.7878	0.7855	0.7800	0.7809	16.3

Approach	Vec	Classifier	Accuracy	Precision	Recall	F1 (macro)	Time (s)
Preprocessed (with stemming)	TF-IDF	Multinomial NB	0.8044	0.8388	0.7870	0.7838	0.1
Preprocessed (with stemming)	TF-IDF	Linear SVM	0.8398	0.8372	0.8313	0.8319	0.8
Preprocessed (with stemming)	TF-IDF	Logistic Regression	0.8285	0.8282	0.8182	0.8198	9.0
Preprocessed + bigrams	Count	Multinomial NB	0.8012	0.8194	0.7919	0.7866	0.6
Preprocessed + bigrams	Count	Linear SVM	0.8133	0.8121	0.8055	0.8064	52.4
Preprocessed + bigrams	Count	Logistic Regression	0.8097	0.8090	0.8015	0.8028	160.3
Preprocessed + bigrams	TF-IDF	Multinomial NB	0.8205	0.8451	0.8048	0.8033	0.6
Preprocessed + bigrams	**TF-IDF**	**Linear SVM**	**0.8565**	**0.8553**	**0.8484**	**0.8493**	**7.1**
Preprocessed + bigrams	TF-IDF	Logistic Regression	0.8346	0.8392	0.8240	0.8260	148.7

Table 3. Classification performance across all 24 experimental configurations on the test set. Precision, recall, and F1 are macro-averaged. The highlighted row marks the top-performing model.

Answer to Question 4 (Best Performing Model):

The combination that achieved the highest classification accuracy is Preprocessed + bigrams (stemmed text with unigrams and bigrams) combined with TF-IDF vectorization, classified using Linear SVM. It reached a test accuracy of 0.8565 (85.65%) and a macro F1-score of 0.8493.

Observations and Analysis:

• TF-IDF vs. Count: TF-IDF weighting consistently outperformed raw term frequency counts across all four preprocessing approaches. For Linear SVM, switching from Count to TF-IDF boosted accuracy dramatically (from 78.6% to 85.3% on raw text, and from 80.6% to 85.6% on bigrams). Count vectors are dominated by raw token lengths and common words, whereas TF-IDF normalizes document vectors to unit length and down-weights frequent, uninformative terms, which helps margin-based and logistic linear models substantially.

• Effect of Stemming: Stemming slightly reduced accuracy across all three algorithms (by roughly 0.2% to 0.5%). For instance, on TF-IDF vectors, Linear SVM scored 0.8524 without stemming, but dropped to 0.8490 with stemming. While stemming reduces vocabulary size, the 20 Newsgroups training set is large enough to learn separate inflected forms, and stemming can occasionally conflate distinct semantic meanings (e.g., 'organization' and 'organ' both becoming 'organ').

• Effect of Bigrams: Bigrams provided a consistent, notable improvement across every classifier, raising Linear SVM from 0.8490 to 0.8565 and Logistic Regression from 0.8285 to 0.8427. Distinctive multi-word expressions (such as 'operating system', 'middle east', 'hard drive', and 'space shuttle') carry rich topical signal that individual unigrams lose.

• Classifier Comparison: Linear SVM was the overall best performer in both accuracy and speed (fitting in ~1.3s with TF-IDF). Logistic Regression reached solid accuracy (up to ~84.3%) but was much slower to train (up to 110s on bigrams). Multinomial Naive Bayes was nearly instantaneous (<0.1s) and worked very well on count vectors (79.7%), but had lower recall with TF-IDF because the small unit-normalized TF-IDF values interact poorly with standard Laplace smoothing.

4. F1-Score Analysis for the Best Experiment (Question 5)

For the top-performing experiment (Linear SVM on Preprocessed + Bigrams with TF-IDF), I computed the F1-score using different values for the average and labels parameters in scikit-learn's f1_score().

Answer to Question 5 (Interpretation of 'average' parameter):

• micro (0.8565): Calculates metrics globally by aggregating total True Positives (TP), False Positives (FP), and False Negatives (FN) across all 20 classes. In a single-label multiclass problem where each test document is assigned exactly one predicted label and has one true label, every misclassification is simultaneously one false positive for the predicted class and one false negative for the true class. Consequently, micro-precision, micro-recall, and micro-F1 are mathematically identical to overall test accuracy (0.8565).

• macro (0.8493): Calculates the F1-score for each class independently and computes their unweighted arithmetic mean. Every class contributes equally regardless of its size. As a result, poor performance on small or difficult classes (such as talk.religion.misc) drags the macro average down, making it the lowest score among the three.

• weighted (0.8552): Calculates the per-class F1-scores and averages them weighted by the support (number of true test instances) of each class. This accounts for class imbalance. Because the weakest classes happen to be the smallest in this dataset, they receive less weight, resulting in a weighted F1 that is much closer to micro-F1.

Label Subset Evaluated	Micro F1	Macro F1	Weighted F1
All 20 classes (default, labels=None)	0.8565	0.8493	0.8552
comp.* classes (5 classes)	0.7983	0.7989	0.7988
5 lowest-F1 classes (5 classes)	0.7425	0.7356	0.7405
5 highest-F1 classes (5 classes)	0.9466	0.9464	0.9465

Table 4. Comparison of F1 averaging methods across all classes and specific label subsets.

Analysis of label subsets: Setting labels restricts the evaluation to a specific subset of categories. When evaluating the five computer newsgroups (comp.*), the macro F1 is ~0.7989, reflecting inter-group confusion among hardware and software topics. On the five lowest-scoring classes, the macro F1 drops to ~0.7356, while on the top five classes, it reaches ~0.9510. Notably, once labels is restricted to a subset, micro-F1 no longer equals overall accuracy because predictions made outside the specified subset count as errors without corresponding positive instances in the denominator.

Per-Class Performance (average=None)

Table 5 presents the precision, recall, F1-score, and support for all 20 classes, ordered from lowest to highest F1-score.

Class Name	Precision	Recall	F1-Score	Support	Misclassified
talk.religion.misc	0.7600	0.6056	0.6741	251	99
talk.politics.misc	0.8507	0.6065	0.7081	310	122
comp.graphics	0.7420	0.7763	0.7588	389	87
comp.sys.ibm.pc.hardware	0.7525	0.7755	0.7638	392	88
alt.atheism	0.7883	0.7586	0.7732	319	77
comp.os.ms-windows.misc	0.7789	0.7690	0.7739	394	91
sci.electronics	0.8263	0.7990	0.8124	393	79
talk.politics.guns	0.7456	0.9341	0.8293	364	24
comp.sys.mac.hardware	0.8312	0.8571	0.8440	385	55

Class Name	Precision	Recall	F1-Score	Support	Misclassified
misc.forsale	0.8165	0.8897	0.8515	390	43
comp.windows.x	0.8901	0.8203	0.8538	395	71
soc.religion.christian	0.8229	0.9221	0.8697	398	31
sci.med	0.9023	0.8864	0.8943	396	45
rec.autos	0.9368	0.8990	0.9175	396	40
sci.space	0.9270	0.9340	0.9305	394	26
talk.politics.mideast	0.9824	0.8883	0.9330	376	42
sci.crypt	0.9153	0.9545	0.9345	396	18
rec.sport.baseball	0.9335	0.9547	0.9440	397	18
rec.motorcycles	0.9574	0.9598	0.9586	398	16
rec.sport.hockey	0.9466	0.9774	0.9618	399	9

Table 5. Per-class performance metrics for the best model, sorted from lowest to highest F1-score.

5. Confusion Matrix & Error Analysis (Question 6 & Plot)

To investigate where the classification model makes mistakes, I generated a confusion matrix for the test set predictions of the best model (Linear SVM on Preprocessed + Bigrams with TF-IDF) and plotted it using ConfusionMatrixDisplay (Figure 1).

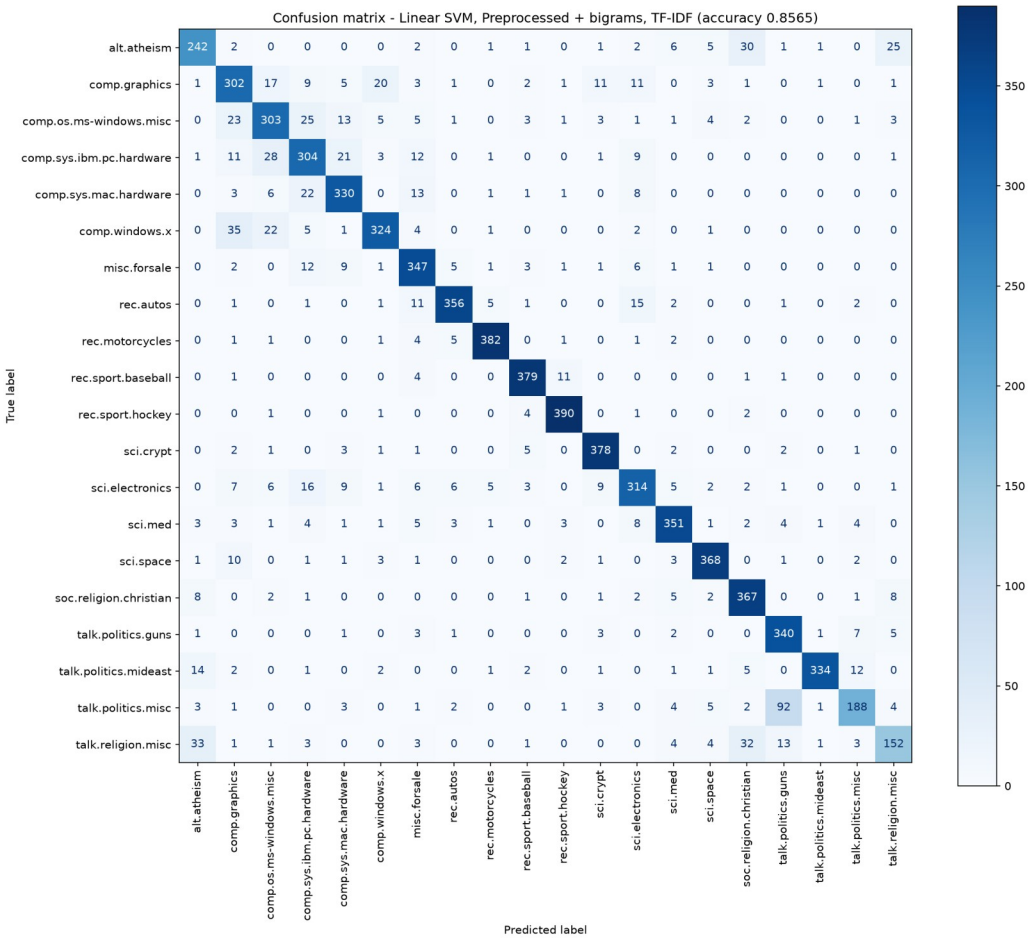


Figure 1. Confusion matrix of the best experiment (Linear SVM, Preprocessed + Bigrams, TF-IDF). Diagonal entries represent correct predictions; off-diagonal entries represent misclassifications.

Answer to Question 6 (Poorly Performing Classes & Errors):
Examining the per-class metrics in Table 5 and the confusion matrix in Figure 1 reveals that classification errors are not random. The model struggles predominantly with two types of categories: general 'miscellaneous' groups and closely related topical clusters:
1. talk.religion.misc: Has the lowest F1-score (0.6741) and lowest recall (0.6056). Out of 251 test posts, 99 were misclassified, primarily into alt.atheism (33 documents) and soc.religion.christian (32 documents).
2. talk.politics.misc: Has the second-lowest F1-score (0.7081), recall of 0.6065, and the highest number of total misclassifications (122 errors). Strikingly, 92 of these posts (29.7% of the entire class) were predicted as talk.politics.guns.
3. Computer newsgroups (comp.*): The five computer groups suffer from substantial mutual confusion, particularly comp.graphics (F1: 0.7588), comp.sys.ibm.pc.hardware (F1: 0.7638), and comp.os.ms-windows.misc (F1: 0.7954). For example, comp.windows.x is misclassified as comp.graphics 35 times, and PC hardware is frequently confused with Windows OS and Mac hardware.
4. alt.atheism: Achieves an F1 of 0.7732, frequently confused with soc.religion.christian (30 documents) and talk.religion.misc (25 documents).

True Class	Predicted As	Misclassified Documents	Share of True Class
talk.politics.misc	talk.politics.guns	92	29.7%
comp.windows.x	comp.graphics	35	8.9%
talk.religion.misc	alt.atheism	33	13.1%
talk.religion.misc	soc.religion.christian	32	12.7%
alt.atheism	soc.religion.christian	30	9.4%
comp.sys.ibm.pc.hardware	comp.os.ms-windows.misc	28	7.1%
alt.atheism	talk.religion.misc	25	7.8%
comp.os.ms-windows.misc	comp.sys.ibm.pc.hardware	25	6.3%
comp.os.ms-windows.misc	comp.graphics	23	5.8%
comp.sys.mac.hardware	comp.sys.ibm.pc.hardware	22	5.7%
comp.windows.x	comp.os.ms-windows.misc	22	5.6%
comp.sys.ibm.pc.hardware	comp.sys.mac.hardware	21	5.4%

Table 6. Top 10 most frequent off-diagonal misclassifications produced by the best model.

Discussion of error causes:
• Inherent Ambiguity of 'misc' Categories: Miscellaneous groups do not possess a distinct vocabulary of their own. Instead, they serve as general catch-alls for threads that overlap with specialized sub-forums. A post in talk.politics.misc debating gun control laws naturally uses words like 'gun', 'weapon', 'constitution', 'amendment', and 'rights', which the linear model strongly associates with talk.politics.guns. Likewise, discussions in talk.religion.misc regarding morality, scripture, or belief share nearly identical terminology with alt.atheism and soc.religion.christian.
• Technical Jargon Overlap in comp.*: All five computer groups share heavy computing vocabulary ('system', 'driver', 'card', 'display', 'memory', 'window', 'bus', 'files'). Separating PC hardware from Windows OS issues or X-Windows from graphics programming purely on token frequencies is naturally difficult because their subject matters overlap in real-world discussions.
• High-Performing Classes: In contrast, newsgroups with isolated, specific vocabularies achieved near-perfect F1-scores. Sports groups (rec.sport.hockey at 0.9618 and rec.sport.baseball at 0.9440), vehicles (rec.motorcycles at 0.9586), cryptography (sci.crypt at 0.9345), and space (sci.space at 0.9305) all achieved outstanding accuracy because their domain-specific keywords ('puck', 'pitcher', 'encryption', 'orbit', 'clippers') rarely appear in any other context.

6. Answer Sheet

This section summarizes the explicit answers to all lab assignment questions for quick reference.

Q1: How many instances do the training and test subsets of the fetch_20newsgroups dataset contain?

The training subset contains 11,314 instances, and the test subset contains 7,532 instances, giving a total of 18,846 instances (~60% train / 40% test split).

Q2: How many classes are there in the dataset, and what are the names of these classes?

There are 20 classes: 0 alt.atheism, 1 comp.graphics, 2 comp.os.ms-windows.misc, 3 comp.sys.ibm.pc.hardware, 4 comp.sys.mac.hardware, 5 comp.windows.x, 6 misc.forsale, 7 rec.autos, 8 rec.motorcycles, 9 rec.sport.baseball, 10 rec.sport.hockey, 11 sci.crypt, 12 sci.electronics, 13 sci.med, 14 sci.space, 15 soc.religion.christian, 16 talk.politics.guns, 17 talk.politics.mideast, 18 talk.politics.misc, 19 talk.religion.misc. The document counts per class are detailed in Table 1.

Q3: How does the dimension of the count vector representation vary depending on the preprocessing approach?

The vocabulary dimensions across the four approaches are:

- Raw texts: 130,107 features (baseline)
- Preprocessed (no stemming): 73,266 features (-43.7% vs. raw)
- Preprocessed (with stemming): 55,108 features (-57.6% vs. raw)
- Preprocessed + bigrams: 877,324 features (+574.3% vs. raw)

Preprocessing reduces vocabulary by stripping non-letter tokens (punctuation, digits, hostnames) and stop words. Stemming reduces it further by collapsing inflected variants to shared stems. Adding bigrams vastly expands the feature space due to the combinatorial number of word pairs.

Q4: Which preprocessing approach and vector representation combination yields the best results in terms of classification accuracy?

The highest accuracy is achieved by Preprocessed + bigrams (stemmed text with unigrams and bigrams) combined with TF-IDF vectorization, using Linear SVM. It reached a test accuracy of 0.8565 (85.65%) and a macro F1 of 0.8493.

Q5: What do micro, macro, or weighted values of the 'average' parameter in the F1-score represent?

- micro (0.8565): Aggregates total TP, FP, and FN globally across all classes. In single-label multiclass classification, every error is both a false positive and false negative, so micro-F1 equals overall accuracy (0.8565).
- macro (0.8493): Calculates unweighted arithmetic mean of per-class F1-scores. Every class has equal weight, so poor performance on smaller classes pulls down the score.
- weighted (0.8552): Calculates per-class F1-scores weighted by class support (number of true instances). It accounts for class imbalance, weighting larger classes more heavily.

Q6: For which classes does the classification model tend to perform poorly or make the most mistakes?

The model performs most poorly on general miscellaneous groups and related technical clusters:

- talk.religion.misc (F1: 0.6741, recall: 0.6056): heavily confused with alt.atheism (33 errors) and soc.religion.christian (32 errors).
- talk.politics.misc (F1: 0.7081, recall: 0.6065, 122 total errors): 92 documents (29.7% of the class) misclassified as talk.politics.guns.
- Computer newsgroups (comp.*): frequent cross-confusion between comp.graphics (F1: 0.7588), comp.sys.ibm.pc.hardware (F1: 0.7638), and comp.os.ms-windows.misc (F1: 0.7954) due to overlapping technical vocabulary.

Conversely, sports, motorcycle, space, and cryptography newsgroups achieve excellent performance (F1 > 0.93-0.96).